

Sistema de Deteccion de Armas mediante Vision Artificial con YOLOv8

Documento tecnico: proceso completo y resultados actuales (v1 a v4)

Plataforma de entrenamiento: NVIDIA RTX 5050 Laptop (Blackwell)

Plataforma de despliegue: Raspberry Pi 5

Fecha: 08/07/2026 16:40

Repositorio: deteccion-de-armas-i.a

Indice

1. Resumen ejecutivo
2. Objetivos del sistema
3. Arquitectura de hardware
4. Stack de software
5. Datasets: evolucion completa (v1 a v4)
6. Modelo y arquitectura YOLO
7. Configuracion del entrenamiento y fine-tuning
8. Sistema de alertas, filtrado y monitoreo
9. Fases ejecutadas (cronologia completa)
10. Resultados actuales
11. Cuantizacion INT8 para Raspberry Pi
12. Despliegue en Raspberry Pi 5
13. Guia: SSH a la Pi desde Visual Studio Code
14. Riesgos identificados y mitigaciones
15. Reproducibilidad y trabajo futuro

1. Resumen ejecutivo

El presente documento describe el desarrollo completo de un sistema de detección en tiempo real de armas blancas (cuchillos) y armas de fuego (pistolas, escopetas, rifles) basado en la arquitectura YOLOv8 de Ultralytics. El sistema fue entrenado en una laptop con GPU NVIDIA RTX 5050 (Blackwell, sm_120) y exportado a ONNX/NCNN para desplegarse en una Raspberry Pi 5 sin necesidad de PyTorch ni Ultralytics.

El proyecto paso por **cuatro iteraciones** mayores. La **v1** uso únicamente Open Images v7 con una clase generica **firearm** y alcanzo un mAP@0.5 de 0.617 en test. La **v2** incorporo datasets de Roboflow Universe, amplio el corpus a 8.656 imagenes curadas y descompuso **firearm** en **handgun** y **long_gun**, alcanzando un **mAP@0.5 de 0.729 en test** (+11.2 pp). Las iteraciones **v3 y v4** abordaron un problema distinto: el modelo detectaba bien pistolas de lado pero fallaba en vistas frontales/POV, y podia **alucinar** armas al ver personas, telefonos o mandos en un entorno tipo stand. Se sumaron +13.902 imagenes (poses diversas + hard negatives de personas/objetos cotidianos) y se aplico un **fine-tune anti-alucinacion** (backbone congelado, LR muy bajo, penalizacion de clasificacion reforzada) partiendo de los pesos v2, sin reentrenar desde cero.

El resultado del fine-tune (v4, evaluado sobre el mismo split de validacion de 865 imagenes que v2): el modelo **nano** (el que se despliega en la Pi) mantuvo su mAP@0.5 practicamente igual (0.8516 vs 0.8530) pero subio su **Precision +2.39 pp** (0.8511 → 0.8750), que es exactamente el objetivo buscado: menos falsas alarmas frente a personas y objetos en el stand. El modelo **small** de referencia se comporto de forma similar (mAP@0.5 -1.10 pp, mismo patron de generalizacion).

El sistema dejo de ser solo un detector: incorpora **filtro geometrico de plausibilidad**, **filtro temporal** (confirmacion multi-frame + suavizado), **umbrales de confianza por clase**, **captura de evidencia** (frames + clips de video con buffer previo/posterior), **notificaciones push por Telegram** y un **dashboard web** para monitoreo remoto desde el celular. El modelo final (YOLOv8n v4) se exporto a ONNX INT8 de **3.25 MB** (-72% vs FP32) y fue **validado en una sesion de camara en vivo** (2026-05-24) donde detecto consistentemente el arma de prueba y genero automaticamente 457 frames y 3 clips de evidencia. A la fecha, el dataset de entrenamiento en disco sigue creciendo (actualmente 27.970 imagenes de train) por el ciclo de retroalimentacion con hard negatives capturados en campo; ese superset aun no cuenta con una re-evaluacion formal.

2. Objetivos del sistema

- Detectar en tiempo real armas blancas (clase **knife**) y armas de fuego diferenciando armas cortas (**handgun**) de armas largas (**long_gun**: rifles, escopetas).
- Reconocer el arma en **poses diversas** (lateral, frontal, POV, angulo CCTV cenital), no solo en la vista de perfil predominante en el dataset original.
- **No alucinar** armas al ver personas, manos, telefonos, mandos, llaves u otros objetos cotidianos tipicos de un escenario de stand/exhibicion con publico.
- Lograr una velocidad de inferencia minima de 10 FPS en Raspberry Pi 5 a resolucion 416x416.
- Mantener el modelo desplegado en pocos MB para permitir actualizaciones remotas eficientes (objetivo cumplido: 3.25 MB en INT8).
- Disparar alertas locales (sonido, log estructurado, frames y clips de video) y remotas (Telegram) cuando se detecte una amenaza con confianza superior al umbral configurado.
- Permitir monitoreo remoto sin pantalla propia mediante un dashboard web accesible desde cualquier dispositivo en la red local.

- Permitir el despliegue completo en la Pi sin necesidad de instalar PyTorch ni Ultralytics, reduciendo la huella a menos de 250 MB de paquetes Python.
- Cerrar el ciclo de mejora continua: los falsos positivos capturados durante pruebas en campo se reintegran como hard negatives en el siguiente entrenamiento.

3. Arquitectura de hardware

3.1 Estacion de entrenamiento

Componente	Especificacion	Rol
Equipo	ASUS TUF Gaming A16 FA608UH	Entrenamiento + inferencia desktop
CPU	AMD Ryzen 7 260 (8 cores / 16 hilos)	DataLoader workers
RAM	32 GB DDR5	Cache de dataset
GPU dedicada	NVIDIA RTX 5050 Laptop, 8 GB VRAM	Entrenamiento CUDA
Arquitectura GPU	Blackwell, capability sm_120	Requiere CUDA 12.8+
GPU integrada	AMD Radeon 780M	Solo display, sin CUDA
Driver NVIDIA	581.80 (CUDA 13.0 runtime)	Verificado al iniciar

3.2 Plataforma de despliegue

Componente	Recomendado
SBC	Raspberry Pi 5 (8 GB RAM)
Almacenamiento	microSD 64 GB clase A2
Refrigeracion	Active cooler oficial (imprescindible bajo carga)
Camara	USB UVC o Pi Camera v3
Sistema operativo	Raspberry Pi OS Bookworm 64-bit
Alimentacion	Fuente oficial 5V/5A

4. Stack de software

El entorno se aísla en un virtualenv Python dentro del repositorio (repo/ .venv). Se forzo la asignacion de la GPU NVIDIA para los ejecutables Python via la clave de registro HKCU\Software\Microsoft\DirectX\UserGpuPreferences.

Paquete	Funcion
torch / torchvision (cu128)	Framework de deep learning con soporte CUDA 12.8
ultralytics	Framework YOLOv8 (entrenamiento + export + val)
fiftyone	Gestion del dataset Open Images v7
roboflow	Descarga de datasets Roboflow Universe (poses de armas)
datasets / huggingface_hub	Descarga de datasets de HuggingFace (PranomVignesh, Subh775, Simuletic, etc.)
opencv-python	Captura de camara, dibujo de bboxes, streaming MJPEG del dashboard
onnx + onnxruntime	Export e inferencia portable + cuantizacion INT8
onnxslim	Optimizacion del grafo ONNX
ncnn	Formato alternativo optimizado para ARM (Raspberry Pi)
pygame	Reproduccion de alertas sonoras (beep sintetico, sin wav externo)

requests	Envio de notificaciones push a la API de Telegram
reportlab	Generacion del presente documento

5. Datasets: evolucion completa (v1 a v4)

5.1 v1 -- Open Images v7 unicamente (2 clases)

Primera version: 2.185 imagenes de Open Images v7 (Knife, Handgun, Shotgun, Rifle) colapsadas en 2 clases (**knife** / **firearm**). La clase generica **firearm** agrupaba pistolas, escopetas y rifles, lo que degrado su rendimiento (mAP@0.5 de 0.49) por la alta varianza visual entre subtipos.

5.2 v2 -- Roboflow + reorganizacion taxonomica (3 clases)

Se sumaron datasets de Roboflow Universe y se establecio la taxonomia final de 3 clases:

- **0 — knife:** cuchillos y armas blancas en general.
- **1 — handgun:** armas cortas (pistolas, revolveres). Empuniadura a una mano.
- **2 — long_gun:** armas largas (rifles, escopetas). Requieren dos manos.

Fuente	Imagenes	Clases originales	Mapeo final
Open Images v7 (FiftyOne)	2.185	Knife, Handgun, Shotgun, Rifle	knife / handgun / long_gun
RF joseph-nelson/pistols	2.971	pistol	handgun
RF rifledetection/rifle-detection-p9slr	3.500*	Rifle	long_gun

* *Sampleadas aleatoriamente desde un total de 13.013 imagenes para mantener balance de clases.*

Particion	Imagenes	Anotaciones knife	Anotaciones handgun	Anotaciones long_gun
Train (85%)	7.357	983	3.593	5.363
Val (10%)	865	93	413	699
Test (5%)	434	51	217	322
Total	8.656	1.127	4.223	6.384

5.3 v3/v4 -- Poses diversas + anti-alucinacion (stand)

Deteccion de dos problemas nuevos al preparar el modelo para exhibirse en un stand con publico: (1) el modelo entrenado en v2 detectaba bien pistolas **de lado** pero fallaba en vistas **frontales / POV / apuntando a camara**; (2) en presencia de personas, manos, telefonos y mandos, podia generar **falsos positivos** (alucinaciones). Aumentar solo con transformaciones geometricas no alcanza: una pistola de lado rotada sigue siendo una pistola de lado, no aporta la vista frontal que falta. Se opto por sumar **datos reales** de esas vistas y de esos objetos confundibles.

Fuente	Imagenes	Tipo
PranomVignesh/HandGuns (HF, con token)	6.882	Mirror Roboflow pistols-p6x3w -- poses variadas
Subh775/WeaponDetection (HF)	5.341	Multi-clase armas
OpenImages stand negatives	846	Personas, moviles, mandos (hard negative)
harshdadiya phone_detection (HF)	605	Moviles puros (hard negative)
Simuletic CCTV Rifle-vs-Umbrella (HF)	120	CCTV cenital + paraguas (hard negative)
Simuletic Handgun-vs-Chips (HF)	110	Bolsa de Doritos vs pistola (hard negative)

Total sumado en la version v4 (final recomendada): **+13.902 imagenes** respecto al train original de v2 (7.357 → 21.259). Los hard negatives se agregan como imagenes de fondo (label vacio) para que el modelo aprenda que esos objetos **no** son armas.

Ademas se implemento un **ciclo de retroalimentacion** (scripts/8_add_hard_negatives.py): los frames capturados como evidencia durante pruebas reales de camara (incluyendo falsos positivos) se reincorporan al split de train como background para el siguiente ciclo de fine-tune.

5.4 Escala actual del dataset (en disco, hoy)

Particion	Imagenes
Train	27.970
Val	865
Test	434
Total	29.269

El train set sigue creciendo por el ciclo de retroalimentacion descrito arriba (27.970 actuales vs 21.259 documentados en el reporte v4). Val y test se mantienen identicos a v2 para que las comparaciones de metricas entre versiones sigan siendo validas. **El superset actual de train (27.970 imgs) aun no tiene una re-evaluacion formal de mAP**; los numeros de la seccion 10 corresponden al ultimo checkpoint evaluado formalmente (v4, 21.259 imgs de train).

5.5 Formato YOLO

Cada imagen NNNNNNN.jpg tiene asociado un archivo NNNNNNN.txt con una linea por objeto detectado:

```
<class_id> <cx> <cy> <ancho> <alto>
```

Donde todas las coordenadas estan normalizadas en el rango [0, 1] respecto al tamano de la imagen, y (cx, cy) es el centro del bounding box. Las imagenes background (hard negatives) usan un archivo .txt vacio.

6. Modelo y arquitectura YOLO

YOLO (You Only Look Once) es una familia de detectores de objetos en una sola pasada. A diferencia de R-CNN que primero propone regiones y luego las clasifica, YOLO predice simultaneamente clase y bounding box procesando la imagen una unica vez, lo que la convierte en la opcion adecuada para inferencia en tiempo real en hardware limitado como la Raspberry Pi.

6.1 Variantes evaluadas

Variante	Parametros	Tamano FP32	Uso en este proyecto
YOLOv8n (nano)	3.0 M	~6 MB / 12 MB ONNX	Despliegue final en Pi5
YOLOv8s (small)	11.1 M	~22 MB	Entrenamiento en laptop (referencia)
YOLOv8m (medium)	26 M	~52 MB	No utilizado

6.2 Estrategia dual de modelos

Se entrenan y afinan **dos modelos en paralelo** sobre el mismo dataset:

- **YOLOv8s** (small) como modelo de referencia con la mejor precision posible. Permite validar la calidad del dataset y servir como benchmark, y correr en la laptop/PC con GPU en tiempo real durante demostraciones.
- **YOLOv8n** (nano) optimizado para tamano y velocidad. Es el modelo que se exporta a ONNX cuantizado INT8 para correr en la Pi.

En la version v4, el nano mantiene un mAP@0.5 casi identico al small (0.852 vs 0.879 en val) con un peso muy inferior y una velocidad de inferencia varias veces mayor en GPU, validando la decision de usarlo como modelo de borde.

7. Configuracion del entrenamiento y fine-tuning

7.1 Hiperparametros base (v2, entrenamiento desde COCO)

Parametro	Valor	Justificacion
epochs	100 (v8s) / 80 (v8n)	Suficiente con dataset ampliado; nano converge antes
batch	8 (v8s) / 24 (v8n)	Ajustado para evitar OOM y errores de cuDNN en Blackwell
workers	4-6	Balance entre velocidad de carga y RAM disponible
imgsz	640 (v8s) / 416 (v8n)	640 estandar en GPU; 416 para FPS en la Pi
optimizer	AdamW	Adaptativo, robusto, estandar moderno
lr0	0.001	Conservador para entrenar desde pesos COCO
patience	30	Early stopping si val no mejora
amp	true	FP16 mixed precision (mitad de VRAM)
mixup / copy_paste	0.20 / 0.50	Mas combinaciones y pegado de armas en otros fondos
degrees / translate / scale	25 / 0.15 / 0.60	Armas en cualquier angulo, posicion y distancia

7.2 Receta de fine-tune anti-alucinacion (v3/v4)

A diferencia del entrenamiento base, el fine-tune **no parte de cero**: continua desde los pesos best . pt de v2 para no perder lo ya aprendido, y usa una receta distinta orientada a reducir falsos positivos:

Parametro	Valor	Por que
punto de partida	best.pt de v2	No reentrenar desde cero: conservar lo ya aprendido
lr0	0.0001	Muy bajo -- ajuste fino, no destruir pesos existentes
lrf / warmup_epochs	0.01 / 3	Decaimiento suave con calentamiento corto
freeze	10 capas (backbone)	Solo se reentrena la cabeza de deteccion
cls (peso de la perdida de clase)	1.0	Penaliza mas fuerte clasificar mal -- menos alucinaciones
perspective / degrees	0.001 / 30	Mas variedad angular para las vistas frontal/POV
close_mosaic	10	Ultimas epochs sin mosaico -- fine-tune limpio
epochs	30 (sin early stopping)	Ciclo corto y controlado de ajuste fino

7.3 Variables de entorno de seguridad

```
PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True
CUDA_VISIBLE_DEVICES=0
KMP_DUPLICATE_LIB_OK=TRUE
```

`expandable_segments:True` es la mitigacion clave del incidente de OOM observado en la version v1 (epoch 70/80). Permite a PyTorch expandir segmentos de VRAM existentes en lugar de fragmentar la memoria con asignaciones nuevas.

8. Sistema de alertas, filtrado y monitoreo

Mas alla del modelo de deteccion, el sistema desplegado incorpora varias capas para hacerlo utilizable en un escenario real (stand, punto de vigilancia sin operador permanente):

8.1 Filtro geometrico de plausibilidad

`utils/plausibility_filter.py` descarta detecciones geometricamente imposibles para cada clase, comparando el area de la caja respecto al frame y su aspect ratio contra rangos esperados:

Clase	Area minima / maxima (frame)	Aspect ratio min / max
knife	0.2% / 45%	1.1 / 18.0
handgun	0.3% / 50%	1.0 / 6.0
long_gun	0.5% / 60%	2.0 / 22.0

8.2 Filtro temporal (anti-flicker)

`utils/temporal_filter.py` no confirma una alerta con una sola deteccion aislada: mantiene una ventana de las ultimas **7 frames** y solo confirma una clase si aparecio en al menos **4 de esos 7**. Las coordenadas del bounding box se suavizan con un promedio movil exponencial (EMA, $\alpha=0.20$) para eliminar saltos visuales. Esto reduce tanto el parpadeo de la caja como las alarmas por detecciones espurias de un unico frame.

8.3 Umbrales de confianza por clase

En vez de un unico umbral global, cada clase tiene su propio umbral de confianza configurado en `config.yaml`: knife 0.40, handgun 0.45, long_gun 0.45 (umbral base 0.30). Esto permite calibrar cada clase de forma independiente segun su tasa de falsos positivos observada.

8.4 Captura de evidencia: frames y clips de video

Ante una deteccion confirmada, `utils/alerts.py` guarda: (1) un beep sintetico generado en runtime (no requiere archivo .wav externo), (2) una linea en `logs/detections.log` (texto legible) y otra en `logs/detections.jsonl` (JSON estructurado para analisis), (3) un frame de evidencia en `logs/frames/`, y (4) un **clip de video** con un buffer en anillo de **5 segundos antes y 10 segundos despues** de la primera deteccion, guardado en `logs/clips/`.

8.5 Notificaciones remotas por Telegram (opcional)

El sistema puede enviar una foto + alerta al chat de Telegram configurado (`telegram_bot_token` / `telegram_chat_id` en `config.yaml`), con un cooldown de 30s entre mensajes para no saturar. Desactivado por defecto hasta configurar el bot.

8.6 Dashboard web de monitoreo remoto

`utils/web_dashboard.py` levanta un servidor HTTP minimalista en un hilo separado (no bloquea la inferencia) que transmite el video en vivo como stream MJPEG y el estado de alertas, accesible desde cualquier dispositivo de la red local en `http://<ip>:8080` -- util para vigilar el sistema desde el celular sin necesitar un monitor conectado.

8.7 Herramientas de mejora continua

- `scripts/8_add_hard_negatives.py`: reintegra los frames capturados (incluyendo falsos positivos) como hard negatives del proximo entrenamiento.
- `scripts/11_capture_finetune_frames.py` + `12_finetune.py`: capturan y etiquetan frames de un escenario/camara especifico para afinar el modelo a ese entorno concreto.
- `scripts/pi5_emulation.py`: emula en la laptop las restricciones de computo de la Pi5 antes de desplegar, para estimar FPS sin necesitar el hardware fisico a mano.

9. Fases ejecutadas (cronologia completa)

Fase	Nombre	Descripcion	Estado
Fase 0	Setup del entorno	Instalacion de Python, virtualenv, PyTorch con soporte CUDA 12.8 (verificado torch.cuda.is_available()==True, RTX 5050 sm_120), instalacion de requirements.txt. Configuracion de preferencia de GPU NVIDIA en el registro de Windows.	Completado
Fase 1	Descarga inicial de Open Images v7 (Olv7)	Primera ejecucion de 1_download_dataset.py. Se identificaron dos bugs: (1) la clase 'Firearm' no existe como leaf class en Olv7; (2) el parametro dataset_dir entraba en conflicto con argumentos internos de FiftyOne. Ambos corregidos, descarga exitosa de 2.185 imagenes en 4 clases.	Completado
Fase 2	Conversion a formato YOLOv11	Mapeo Olv7 -> 2 clases (knife/firearm), generacion de splits y data.yaml.	Completado
Fase 3	Entrenamiento YOLOv11	500 epochs configuradas, batch 12, workers 8. Crash con CUDA OOM en epoch 70/80 por fragmentacion de VRAM. best.pt se conservo.	Completado con incidente
Fase 4	Evaluacion v1	Test (516 imgs): mAP@0.5 = 0.617, mAP@0.5:0.95 = 0.474, P = 0.83, R = 0.65. Bajo rendimiento en la clase generica firearm (0.49).	Completado
Fase 5	Descarga de datasets	Robertson/p9slr (2.971 imgs) y rifledetection/rifle-detection-p9slr (sampleado a 3.500 imgs). 2b_merge_datasets.py re-mapea a 3 clases y genera particion 85/10/5. Resultado: 8.656 imagenes finales.	Completado
Fase 6	Reentrenamiento YOLOv11	100 epochs, mAP@0.5 test = 0.729 (+11.2pp vs v1). handgun paso de 0.49 (agregada) a 0.80 (separada) -- la mejora individual mas grande.	Completado
Fase 7	Entrenamiento YOLOv11	80 epochs con cache en RAM. mAP@0.5 val ~0.85. Export ONNX FP32 (11.6 MB) + INT8 cuantizado (3.25 MB, -72%).	Completado
Fase 8	Publicacion inicial en GitHub	Commit con scripts, config v2 y documentacion tecnica (version inicial del presente documento).	Completado
Fase 9	Sesion de captura + has scripts	8, 9, 10 y 11. Se capturan frames propios de camara, se agregan negativos extra de Open Images (personas/objetos sin fiftyone, evitando su crash de MongoDB en Windows) y se descargan mas imagenes de armas directamente de los CSV de Olv7.	Completado

Fase 10	Diagnostico: fallas en procesamiento de la v2	Se identifican tres problemas de la v2: falla con pistolas de frente/POV y puede confundir personas/telefonos/mandos con armas en escenarios con publico (stand). Se descarta el aumento por transformaciones geometricas como solucion suficiente.	Completado
Fase 11	Descarga masiva de datasets	scripts 13, 14, 15, 16 y 17. Hard negatives (N3/gg) (PranomVignesh/HandGuns, Subh775/WeaponDetection, phone_detection, CCTV Rifle-vs-Umbrella, Handgun-vs-Chips) + negativos de personas/objetos de OpenImages. +13.902 imagenes.	Completado
Fase 12	Fine-tune anti-alucinacion	script 17. Se finetune desde best.pt v2 con backbone congelado, lr0=0.0001, cls=1.0, 30 epochs. Primera version intermedia (v3), sin el dataset PranomVignesh completo.	Completado
Fase 13	Reintegracion completa	scripts 18 y 19. Reintegracion PranomVignesh completo (con token HF, ~8.9k imgs vs subset previo), reentrena nano y small, y cada uno se valida + exporta (ONNX/NCNN) + reporta automaticamente (script 18). Genera docs/finetune_results.md.	Completado
Fase 14	Funcionalidades de sistema	Se crea: alertas/flags, dashboard, TimePlausibility_filter.py, utils/temporal_filter.py, utils/web_dashboard.py, umbrales por clase, clips de video con buffer y notificaciones por Telegram (opcional). Se actualiza config.yaml al modelo v4.	Completado
Fase 15	Validacion en vivo con cliente	Sesion (2023-05-24) prueba con el modelo nano v4: deteccion consistente del arma de prueba durante varios minutos, 457 frames de evidencia y 3 clips de video generados automaticamente, logs de texto y JSON poblados end-to-end.	Completado
Fase 16	Ciclo de retroalimentacion	Los frames capturados en pruebas de campo se reincorporan como hard negatives (script 8). El train set en disco crecio de 21.259 (v4 reportado) a 27.970 imagenes actuales. Pendiente: re-evaluacion formal sobre este superset.	En curso

10. Resultados actuales

10.1 Progresion historica en el split de test (434 imgs, sin tocar desde v1)

Metrica	v1 (Olv7, 2 clases)	v2 (Olv7+RF, 3 clases)	Delta
Imagenes de train	1.500	7.357	+391%
mAP@0.5 (test)	0.617	0.729	+11.2 pp
mAP@0.5:0.95 (test)	0.474	0.589	+11.5 pp
Precision	0.83	0.868	+3.8 pp
Recall	0.65	0.745	+9.5 pp

10.2 v2 por clase (split test, 589 instancias)

Clase	Instancias	Precision	Recall	mAP@0.5	mAP@0.5:0.95
all	589	0.868	0.745	0.729	0.589
knife	51	0.760	0.745	0.724	0.655
handgun	217	0.951	0.806	0.802	0.649
long_gun	321	0.894	0.682	0.660	0.464

Hallazgo clave del salto v1→v2: la clase handgun paso de 0.49 (agregada como firearm generica) a 0.80 (separada). La descomposicion taxonomica fue la mejora individual mas grande de esa etapa.

10.3 Fine-tune anti-alucinacion v4 vs v2 (split val, 865 imgs, 1.205 instancias)

Esta comparacion usa el split de **validacion** (no el de test de la seccion 10.1) y es la que se utilizo para decidir si aceptar el fine-tune v4. Nota: este val set sigue dominado por la distribucion original (pistolas de lado); los modelos v4 reparten capacidad entre esa distribucion vieja y las nuevas (frontal/POV/hard-negs), por lo que aqui se ve una bajada esperable de mAP. La ganancia real (menos falsas alarmas y mejor deteccion frontal) se observa en el despliegue, no en este val estatico.

Modelo	Param (M)	imgsz	mAP@50	mAP@50-95	Precision	Recall
nano v2 (original)	3.01	416	0.8530	0.6208	0.8511	0.7757
nano v4 (final, en Pi)	3.01	416	0.8516	0.6105	0.8750	0.7585
small v2 (original)	11.13	640	0.8901	0.6476	0.8858	0.8249
small v4 (final, PC)	11.13	640	0.8791	0.6311	0.8787	0.7924
Modelo	Delta mAP@50		Delta mAP@50-95		Delta Precision	Delta Recall
nano (v4 vs v2)	-0.14 pp		-1.03 pp		+2.39 pp	-1.73 pp
small (v4 vs v2)	-1.10 pp		-1.65 pp		-0.71 pp	-3.26 pp

Lectura: en el modelo nano (el que va a la Pi), el mAP@0.5 se mantuvo practicamente igual (-0.14pp, dentro de ruido) mientras que la **Precision subio +2.39 pp** -- el modelo se volvio mas exigente para decir "hay arma", que es exactamente el objetivo anti-alucinacion para el stand. El Recall bajo -1.73pp

(algun verdadero positivo en pose rara podria escaparse), una compensacion aceptada porque el objetivo prioritario es evitar falsas alarmas frente al publico. Ambas versiones (v2 y v4) quedan exportadas y disponibles, por lo que revertir a v2 es inmediato si en campo v4 se comportara peor.

10.4 Velocidad de inferencia (GPU RTX 5050, val)

Modelo	Preprocess	Inferencia	Postprocess
nano v4 (416px)	~0.4 ms	~1.1-1.6 ms	~0.8-0.9 ms
small v4 (640px)	~0.8-0.9 ms	~5.8-6.2 ms	~0.8 ms

10.5 Validacion en campo con camara en vivo (2026-05-24)

Mas alla de las metricas de benchmark, se corrio una sesion real con el modelo nano v4 sobre la camara en vivo para verificar el pipeline completo (deteccion → filtro temporal → alerta → evidencia), no solo el modelo aislado:

- El sistema detecto **consistentemente** el arma de prueba (handgun) frame a frame durante toda la sesion, confirmando el filtro temporal (≥ 4 de 7 frames).
- Se generaron automaticamente **457 frames** de evidencia en `logs/frames/`.
- Se generaron **3 clips de video** con buffer de 5s antes / 10s despues en `logs/clips/`.
- `logs/detections.log` y `logs/detections.jsonl` quedaron poblados con timestamp, numero de frame y clase por cada deteccion confirmada.

Esta prueba valida el sistema de punta a punta (no solo el mAP del modelo), incluyendo la cadena de alertas y captura de evidencia que se usara en el despliegue final.

11. Cuantizacion INT8 para Raspberry Pi

La cuantizacion convierte los pesos del modelo de 32 bits flotantes (FP32) a 8 bits enteros (INT8). Esto reduce el tamaño del archivo en ~72% y acelera la inferencia en hardware ARM, con una pérdida típica de mAP menor al 2%.

11.1 Proceso aplicado

Se utilizó `onnxruntime.quantization.quantize_static()` con calibración estática sobre el modelo final **yolov8n_v4_pose_negs**:

- Set de calibración: imágenes seleccionadas aleatoriamente del split de train
- Formato: QDQ (Quantize-DeQuantize), per-channel
- Pesos: QInt8 | Activaciones: QUInt8
- Método de calibración: MinMax
- Formatos adicionales exportados: NCNN (param+bin), alternativa optimizada para ARM

11.2 Resultado (modelo desplegado hoy)

Modelo	Formato	Tamaño	Reducción
yolov8n_v4_fp32.onnx	FP32	11.61 MB	baseline
yolov8n_v4_int8.onnx	INT8 estatico QDQ	3.25 MB	-72%
yolov8n_v4 (best_ncnn_model)	NCNN FP32	~11.5 MB (param+bin)	alternativa ARM

12. Despliegue en Raspberry Pi 5

12.1 Software a instalar en la Pi

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y python3-pip python3-venv libatlas-base-dev libopenblas-dev
python3 -m venv ~/weapons-env
source ~/weapons-env/bin/activate
pip install opencv-python onnxruntime numpy pyyaml pygame requests
```

Total instalado: ~250 MB. **No** se instala PyTorch ni Ultralytics.

12.2 Archivos a copiar desde la laptop

Archivo	Funcion	Tamaño
models/export/yolov8n_v4_int8.onnx	Modelo cuantizado INT8 (final)	3.25 MB
config.yaml	Umbral por clase, alertas, dashboard, Telegram	300 B
utils/	Filtros, visualización, alertas, dashboard	~30 KB
scripts/7_inference_pi.py	Loop de inferencia	5 KB

12.3 Comando scp para transferir

```
scp models/export/yolov8n_v4_int8.onnx pi@<ip-pi>:~/weapons/best.onnx
scp config.yaml pi@<ip-pi>:~/weapons/
scp -r utils pi@<ip-pi>:~/weapons/
scp scripts/7_inference_pi.py pi@<ip-pi>:~/weapons/
```

12.4 Performance esperado

Resolucion	Modelo	FPS esperados en Pi 5
416x416	YOLOv8n FP32 ONNX	4 - 7
416x416	YOLOv8n INT8 ONNX (desplegado)	12-18 (objetivo)
320x320	YOLOv8n INT8 ONNX	18 - 25
416x416	YOLOv8n NCNN FP16	15 - 22

El dashboard web (seccion 8.6) puede activarse tambien en la Pi para monitoreo remoto; considerar bajar fps_limit del stream MJPEG si el ancho de banda de la red local es limitado.

13. Guia: SSH a la Pi desde Visual Studio Code

Esta seccion documenta el flujo completo para administrar y desarrollar en la Raspberry Pi 5 desde Visual Studio Code en la laptop, usando la extension Remote-SSH.

13.1 Setup inicial de la Pi

- Flashear microSD con **Raspberry Pi Imager** (Pi OS Lite 64-bit).
- En el menu avanzado del Imager (icono de engranaje), configurar:
 - - Hostname: raspberrypi
 - - Habilitar SSH (con autenticacion por password o llave)
 - - Usuario y password (ej. pi / tu password)
 - - Configurar WiFi (SSID + password)
- Insertar la SD en la Pi, conectar fuente y esperar 60-90 segundos al primer boot.

13.2 Encontrar la IP de la Pi

Desde la laptop, opcion mas rapida (ping mDNS):

```
ping raspberrypi.local
```

Si no responde, escanear la red:

```
# Windows (PowerShell)
arp -a | findstr 'b8-27-eb\|dc-a6-32\|e4-5f-01\|2c-cf-67'

# Linux/Mac
nmap -sn 192.168.1.0/24
```

13.3 Configurar SSH key (recomendado)

Desde la laptop, generar par de llaves si no existe:

```
ssh-keygen -t ed25519 -C 'pi-deploy'
```

Copiar la llave publica a la Pi:

```
# Windows PowerShell
type $env:USERPROFILE\.ssh\id_ed25519.pub | ssh pi@<ip> 'cat &&
~/.ssh/authorized_keys'

# Linux/Mac
ssh-copy-id pi@<ip>
```

A partir de aqui no se pide password al conectar.

13.4 Editar ~/.ssh/config en la laptop

Esto permite conectarse simplemente con ssh pi5:

```
Host pi5
HostName 192.168.1.42 # reemplazar por la IP real
User pi
IdentityFile ~/.ssh/id_ed25519
ServerAliveInterval 60
```

13.5 Instalar la extension Remote-SSH en VS Code

- Abrir Visual Studio Code en la laptop.
- Ir a Extensions (Ctrl+Shift+X).
- Buscar e instalar: **Remote - SSH** (publisher: ms-vscode-remote).
- Opcionalmente instalar tambien **Remote - SSH: Editing Configuration Files**.

13.6 Conectarse a la Pi desde VS Code

- Abrir Command Palette: Ctrl+Shift+P.
- Escribir: **Remote-SSH: Connect to Host...**
- Seleccionar pi5 (o el alias configurado).
- Se abre una nueva ventana de VS Code conectada a la Pi.
- La primera vez descarga el VS Code Server en la Pi (~80 MB, una sola vez).
- Esquina inferior izquierda mostrara: SSH: pi5 (verde).

13.7 Estructura de carpetas en la Pi

```
/home/pi/  
weapons-env/ # virtualenv  
weapons/  
best.onnx # modelo INT8 (3.25 MB)  
config.yaml  
utils/  
visualization.py  
alerts.py  
plausibility_filter.py  
temporal_filter.py  
web_dashboard.py  
7_inference_pi.py  
logs/  
detections.log  
detections.jsonl  
frames/ # capturas de evidencia  
clips/ # clips de video (pre/post buffer)
```

13.8 Subir archivos: dos opciones

Opcion A — arrastrar y soltar en VS Code:

- Una vez conectado, abrir el explorador de archivos (Ctrl+Shift+E).
- Navegar a /home/pi/weapons.
- Arrastrar archivos desde el explorador del SO directamente a la ventana.

Opcion B — via terminal con scp:

```
scp models/export/yolov8n_v4_int8.onnx pi5:~/weapons/best.onnx  
scp config.yaml pi5:~/weapons/  
scp -r utils pi5:~/weapons/  
scp scripts/7_inference_pi.py pi5:~/weapons/
```

13.9 Ejecutar inferencia desde la terminal de VS Code

- En VS Code conectado a la Pi, abrir terminal: Ctrl+\`.

- Activar el venv:

```
source ~/weapons-env/bin/activate
```

- Ejecutar inferencia:

```
cd ~/weapons && python 7_inference_pi.py --weights best.onnx
```

- Para detener: Ctrl+C.

13.10 Servicio systemd para arranque automatico

Para que la deteccion arranque sola cada vez que se enchufa la Pi, crear /etc/systemd/system/weapons.service:

```
[Unit]
Description=Weapons Detection
After=network.target

[Service]
User=pi
WorkingDirectory=/home/pi/weapons
ExecStart=/home/pi/weapons-env/bin/python 7_inference_pi.py --weights best.onnx
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target
```

Activar:

```
sudo systemctl daemon-reload
sudo systemctl enable weapons
sudo systemctl start weapons
sudo systemctl status weapons # verificar
```

13.11 Comandos utiles desde la terminal

Comando	Funcion
journalctl -u weapons -f	Ver logs en vivo del servicio
sudo systemctl restart weapons	Reiniciar deteccion
htop	Monitor de CPU y RAM
vcgencmd measure_temp	Temperatura del SoC
v4l2-ctl --list-devices	Listar camaras detectadas
uptime	Tiempo encendido + carga
df -h /	Espacio en disco

13.12 Troubleshooting comun

Problema	Solucion
Camara no detectada (cv2.VideoCapture(0))	Probar otros indices (1, 2). Verificar permisos: sudo usermod -aG video pi
FPS muy bajos (<5)	Verificar throttling: vcgencmd get_throttled. Si distinto de 0x0, problema termico/voltaje
No se reproduce sonido	Verificar audio: aplay -l. Configurar default device en ~/.asoundrc
VS Code Remote-SSH no conecta	Borrar ~/.vscode-server en la Pi y reintentar. Verificar version de SSH (>=8.0)
systemctl status reporta failed	Ver journalctl -u weapons -n 50 para detalle
Dashboard web no accesible desde el celular	Verificar que celular y Pi esten en la misma red/VLAN; revisar firewall (sudo ufw allow 80)
Poca memoria al inferir	Cerrar X11/desktop si se uso Pi OS Desktop (preferir Lite)

14. Riesgos identificados y mitigaciones

Riesgo	Impacto	Mitigacion aplicada
OOM de VRAM al final del entrenamiento en epoch 70/80	Crash en epoch 70/80	batch 12->8, workers 8->4, expandable_segments:True
Fragmentacion progresiva de VRAM	OOM progresivo	PYTORCH_CUDA_ALLOC_CONF configurada
Modelo v2 solo entrenado con pistola de perfilistas frontales/POV	Falta de perfiles frontales/POV	66.9k imgs de PranomVignesh/HandGuns con poses variadas (v3/
Riesgo de alucinacion en stand (personas, armas, frentales)	Falsos positivos	publicar negatives especificos (846+605+120+110 imgs) + fine-tune c
Fine-tune agresivo podria olvidar lo aprendido (catastrofe forgeting)	Agredido (catastrofe forgeting)	y bajo (0.0001) + freeze de backbone (10 capas) + partir de
Deteccion aislada de un solo frame	Alarma de un solo frame	Temporal: confirmacion >=4/7 frames + suavizado EMA
Cajas geometricamente imposibles	Falsos positivos	plausibility_filter.py con rangos de area/aspect-ratio por clase
Dataset de train sigue creciendo sin nuevos de un APr report	Nuevos de un APr report	(23.970+211+258+11) y 10.3; re-evaluar a
Clase minoritaria knife (proporcionalmente a las otras clases)	Baja cantidad de ejemplos	_paste augmentation 0.5 + umbral de confianza especifico ma
GPU NVIDIA Blackwell sm_120 incompatible con CuDNN 8.1	Crash en epoch 1	PyTorch instalado contra cu128 explicitamente
batch 32 dispara cuDNN error en yolo	Crash en epoch 1	Reducido a batch 24 (base) / 16 (export/finetune)
Pistola de prueba (replica) fuera de distribucion (color, textura)	Falsos positivos	Imprimir/usar en negro mate; bajar threshold si es necesario; valida

15. Reproducibilidad y trabajo futuro

15.1 Para reproducir todo desde cero

Ver el archivo README.md / README.txt en la raiz del repositorio.

15.2 Trabajo futuro sugerido

- Re-evaluar formalmente el modelo sobre el superset actual de train (27.970 imgs) para confirmar que los numeros de la seccion 10.3 siguen vigentes.
- Construir un split de validacion propio con escenas de stand reales (personas + objetos cotidianos + arma de prueba) para medir directamente la tasa de falsos positivos person→handgun, en vez de inferirla solo del val set historico.
- Ampliar dataset con mas ejemplos de armas en contextos reales (manos, ropa, interiores, exteriores) mas alla de las fuentes ya integradas.
- Probar variantes YOLOv8 mas grandes (m, l) en la laptop como modelo "teacher" para destilacion al nano.
- Implementar tracking entre frames (ByteTrack/BoTSORT) ademas del filtro temporal actual, para reducir aun mas alertas duplicadas sobre el mismo objeto.
- Optimizar y medir el modelo NCNN en la Pi5 real y comparar FPS contra ONNX Runtime en hardware fisico (hoy solo se cuenta con estimaciones/emulacion).
- Considerar deteccion en stream RTSP (camaras IP) ademas de USB/Pi Camera.
- Activar y probar en campo la notificacion por Telegram y el dashboard web en la Raspberry Pi 5 real (hoy validados solo en la laptop).

